SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE **COURSE CODE: CSA12**

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 5 Total Marks: 25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I **V JEEVANANTHAM** (Name / Reg No: **192421234**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *V Jeevanantham*

ANSWERS

Name: V JEEVANANTHAM Reg No: 192421234

Question 1

A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

Understanding the Problem

A surveillance system with multiple cameras generates continuous, high-volume video data that must move between I/O devices (cameras), memory (buffers/storage), and the CPU (for processing and threat detection). During peak hours, the sheer volume of simultaneous data transfers causes congestion, leading to lag in threat detection — a classic case of I/O and memory bandwidth becoming the bottleneck rather than raw CPU speed.

How CPU–Memory–I/O Interaction Causes Lag

Each camera feed requires DMA (Direct Memory Access) or CPU-mediated transfer into memory before processing. If the CPU has to individually service every I/O request through polling or unbuffered interrupts, it spends more time managing transfers than analyzing data. Memory access delays in fetching and storing frames compound with CPU wait states, increasing the effective cycle time per instruction and slowing the detection pipeline.

Role of Bus Structures

In a **single-bus architecture**, the CPU, memory, and I/O devices share one common pathway. Under heavy load with many cameras streaming simultaneously, only one transfer can occur at a time, causing serialization delays and bus contention — this is the primary cause of lag in this scenario. In a **multi-bus architecture**, separate buses (a dedicated I/O bus, memory bus, and system bus) allow simultaneous transfers, for example one camera's data moving to memory while the CPU reads another buffer at the same time. This parallelism drastically reduces contention and lag.

Improvements via Instruction Execution Cycle

Pipelining the fetch–decode–execute cycle allows overlapping of instruction stages, increasing the throughput of the detection algorithm. Interrupt-driven I/O combined with DMA frees the CPU from managing every transfer directly, letting it focus on analysis. Cache optimization further reduces repeated memory fetches for frequently used frame-processing instructions.

Conclusion

Adopting a multi-bus architecture with DMA-based I/O, combined with a pipelined instruction cycle and effective caching, resolves the bottleneck and enables real-time threat detection even under peak load.

Question 2

A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

Understanding the Problem

The system executes millions of instructions per second but suffers delays because of a high CPI (Cycles Per Instruction), driven by frequent memory accesses and branch instructions. Execution time is governed by the relation: Execution Time = Instruction Count × CPI × Clock Cycle Time. Even with a fast clock, a high CPI inflates total execution time, causing transaction delays.

Impact of the Fetch–Decode–Execute Cycle

In the fetch stage, frequent memory access such as fetching market data and order records causes memory-bound stalls, since memory is slower than the CPU. In the decode stage, complex instructions involving conditional logic for trade decisions take longer to decode. In the execute stage, branch instructions used for buy/sell conditions disrupt the pipeline, causing branch mispredictions and pipeline flushes, which add extra cycles per instruction.

Methods to Reduce CPI and Improve Performance

1. Pipelining: overlap the fetch, decode, and execute stages of successive instructions to increase instruction throughput.
2. Branch prediction: dynamic branch predictors reduce pipeline stalls caused by conditional trading logic.
3. Cache hierarchy (L1/L2/L3): reduces the cost of frequent memory accesses by keeping recently used stock data closer to the CPU.
4. Superscalar execution: issue multiple instructions per cycle to hide latency from memory-bound operations.
5. Reducing memory access frequency: using registers more effectively for frequently used values, such as the current stock price, instead of repeated memory reads.

Conclusion

By combining pipelining, branch prediction, and a strong cache hierarchy, the effective CPI can be reduced significantly, which directly lowers total execution time and removes transaction delays — a critical requirement for real-time trading systems.

Question 3

An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

Understanding the Problem

An embedded temperature control system reads sensor data as input and drives actuators as output based on processed values. Occasional errors in temperature readings are traced to improper use of addressing modes in the program, meaning the CPU may be reading or writing the wrong memory location, register, or data value.

Role of Addressing Modes in Data Access

In the 8085, addressing modes determine how operands are located. Immediate addressing places data directly within the instruction (for example, MVI A, 05H); misuse here can hardcode wrong constant sensor thresholds. Direct addressing specifies the operand address directly (for example, LDA 2050H); an incorrect address here would read the wrong memory location for temperature data. Register addressing keeps data in a CPU register (for example, MOV A, B); if the wrong register holds stale sensor data, incorrect values get processed. Register indirect addressing holds the address in a register pair (for example, LDAX B) and is commonly used for pointer-based access to sensor buffers;

an un-updated pointer would fetch old or wrong data. Implied addressing has the operand implied by the opcode (for example, CMA); misuse is less likely but can still affect flag-dependent logic.

Possible Causes of Errors

Using direct addressing with a fixed memory address instead of dynamically updating pointers through register indirect addressing as new sensor values arrive; reusing a register that has not been refreshed with the latest sensor input, causing the actuator to respond to a stale value; and incorrect immediate values used for calibration thresholds.

Ensuring Accurate and Reliable Operation

Register indirect addressing should be used for reading sequential sensor data streams, ensuring pointers are correctly incremented or updated after each read. Memory addresses used in direct addressing instructions, especially for critical control registers, should be validated. The instruction set should be used carefully — for example, IN/OUT instructions for correct port-based sensor and actuator communication, and flag-based conditional jumps such as JZ and JNZ should correctly reflect updated data states after each ALU operation. Periodic re-validation or checksum routines can further help detect corrupted addressing before the actuator responds.

Conclusion

Correct and consistent use of addressing modes — particularly register-indirect addressing for dynamic sensor data and validated direct addressing for control constants — combined with disciplined instruction sequencing, ensures accurate and reliable temperature control operation.
